

AI 에게 일을 맡기는 법

비개발자를 위한 AI 에이전트 · 바이브 코딩 · 업무자동화 실전
가이드

박종호 지음

2026 년 9 월 개정판

책을 시작하기 전에

이 책은 생성형 AI 를 “잘 질문하는 도구”로만 다루지 않는다. 핵심은 AI 에게 실제 일을 맡기기 위해 업무를 구조화하고, 자료와 도구를 연결하고, 결과를 검증하는 방법이다. 독자는 코딩을 모르는 실무자와 경영자를 기본 독자로 삼았다. 따라서 수학적식이나 모델 구조보다 업무 흐름, 판단 기준, 실습 결과물을 중심으로 설명한다.

초고는 KOTRA 글로벌 AI 리더 양성 과정의 「AI 에이전트 실습 교육」이라는 문제 의식과 공개 확인 가능한 유사 교육 커리큘럼을 참고해 독자적으로 재구성했다. 원 Padlet 은 외부 환경에서 게시물 전체가 정상 로딩되지 않아, 특정 슬라이드나 문장을 그대로 옮기지 않았다. 대신 “물어보는 AI 에서 실행하는 AI 로”, 코딩 에이전트 설정, 나만의 에이전트 구현, 벤치마킹과 개선, RAG 와 업무 재설계라는 공개 커리큘럼의 큰 흐름을 바탕으로 책의 서술을 새로 작성했다.

AI 제품과 화면은 빠르게 바뀐다. 그래서 이 책은 버튼 위치를 외우게 하기보다 오래 가는 원리를 가르친다. 도구 이름이 바뀌어도 목표-입력-판단-도구-검증-실행이라는 구조는 남는다. 2026 년 9 월 기준의 도구와 사례는 “현재의 예”로 읽고, 실제 적용 시에는 각 서비스의 최신 정책과 보안 기준을 반드시 확인하기 바란다.

저작권과 활용 원칙

이 책은 공개 커리큘럼과 저자의 독자적 해설·실습 설계를 결합한 실무서다. 특정 강의자료의 도표·문장·실습 파일을 복제하지 않는다. 회사 자료, 고객 데이터, 개인정보를 AI 서비스에 입력할 때에는 소속 조직의 보안 정책과 관련 법규를 우선한다.

차례

PART 1 AI 를 도구가 아니라 동료로 보기

- 01 AI 에이전트 시대, 무엇이 달라졌나 8
- 02 내 일을 AI 가 이해하게 만드는 법 11
- 03 좋은 프롬프트보다 좋은 업무 지시서 14

PART 2 일을 워크플로로 바꾸기

- 04 업무를 입력-판단-실행으로 쪼개라 18
- 05 컨텍스트 엔지니어링: AI 에게 필요한 것만 보여주기 21
- 06 사람의 승인을 설계하는 법 24

PART 3 코딩을 몰라도 만드는 시대

- 07 바이트 코딩의 원리 28
- 08 Codex 와 코딩 에이전트로 첫 앱 만들기 31
- 09 GitHub 로 AI 작업을 관리하기 34

PART 4 회사 자료와 데이터를 연결하기

- 10 RAG: 우리 회사 문서를 읽는 AI 38
- 11 스프레드시트와 데이터를 분석하는 AI 41
- 12 API 와 함수 호출: AI 에게 손과 발 달기 44

PART 5 에이전트를 실제 업무에 배치하기

- 13 리서치 에이전트 48
- 14 영업·고객관리 에이전트 50
- 15 콘텐츠·보고서 에이전트 52
- 16 멀티 에이전트: AI 팀을 운영하는 법 54

PART 6 연결과 자동화의 완성

- 17 MCP: AI 와 도구를 표준으로 연결하기 57

18 반복 업무를 스케줄과 조건으로 자동화하기 59

19 AI 품질·보안·거버넌스 61

PART 7 90 일 안에 회사 AI 시스템 만들기

20 AI 전환 과제의 우선순위 정하기 64

21 90 일 실행 로드맵 66

22 AI 시대의 리더십: 더 많이 일하지 말고 더 잘 설계하라 68

프롤로그

AI 를 잘 쓰는 사람보다, 일을 잘 맡기는 사람이 된다

생성형 AI 가 처음 등장했을 때 많은 사람은 “무엇을 물어보면 좋은 답을 받을까”에 관심을 가졌다. 몇 년이 지나자 질문의 수준보다 더 중요한 차이가 생겼다. 어떤 사람은 여전히 문장 하나를 고쳐 달라고 하고, 어떤 사람은 자료를 모으고 분석하고 초안을 만들고 검수한 뒤 정해진 형식으로 결과물을 내는 전체 과정을 AI 에게 맡긴다. 같은 모델을 써도 생산성의 차이가 커지는 이유다.

이 책의 출발점은 단순하다. AI 를 사람처럼 믿으라는 뜻이 아니다. 오히려 반대다. 사람에게 일을 맡길 때 목표와 자료, 마감, 품질 기준을 분명히 하듯 AI 에게도 일을 구조화해서 맡겨야 한다. 무엇을 자동화하고 무엇이 사람이 승인할지, 어떤 자료를 참조하고 어떤 자료는 절대 보내지 않을지, 결과가 틀렸을 때 어떻게 되돌릴지를 설계해야 한다.

비개발자에게 이 변화는 특히 중요하다. 코딩 에이전트가 자연어 지시를 받아 파일을 만들고, 데이터를 정리하고, 작은 웹앱을 구축하는 수준까지 왔다. 이제 “개발을 할 줄 아느냐”보다 “무엇을 만들지 정확히 설명할 수 있느냐”가 더 큰 병목이 되는 순간이 많다. 이 책은 그 설명 능력을 업무 설계 능력으로 확장하는 안내서다.

각 장에는 개념보다 결과물을 남기는 실습을 배치했다. 한 장을 읽고 나면 프롬프트 하나가 아니라 업무 지시서, 워크플로, 데이터 사전, 에이전트 설계서, 자동화 체크리스

트 같은 자산이 생긴다. 끝까지 따라가면 최소 하나의 실제 업무 에이전트를 운영할 수 있는 90일 계획을 완성하게 된다.

이 책의 사용법

읽기만 하지 말고 각 장의 실습 산출물을 한 폴더에 모아라. 22장을 끝냈을 때 그 폴더가 바로 당신 회사의 첫 번째 AI 운영 매뉴얼이 된다.

PART 1 AI 를 도구가 아니라 동료 로 보기

생성형 AI 를 질문 도구에서 실행 시스템으로 바라보기 위한 기본 관점을 세운다.

CHAPTER 01

AI 에이전트 시대, 무엇이 달라졌나

챗봇은 답을 만들지만, 에이전트는 목표를 향해 여러 단계를 실행한다.
이 차이를 이해해야 AI 활용의 다음 단계가 보인다

질문에 답하는 AI 에서 일을 끝내는 AI 로

전통적인 챗봇 사용은 입력과 출력 사이가 짧다. “이 메일을 자연스럽게 고쳐줘”라고 입력하면 답장을 받고 끝난다. 에이전트 방식은 다르다. “지난 전시회 바이어 중 30 일 이상 후속 연락이 없는 사람을 찾아 관심 제품을 확인하고, 거래 단계에 맞는 후속 메일 초안을 만들라”처럼 목표를 준다. 그러면 AI 는 데이터를 찾고, 조건을 판단하고, 필요한 도구를 사용하며 중간 결과를 누적한다.

에이전트의 본질은 자율성이 아니라 작업의 연속성이다. 한 번의 응답에 모든 것을 우겨 넣지 않고, 목표를 여러 단계로 나누어 상태를 유지하며 다음 행동을 선택한다. 이때 자율성은 무제한 자유가 아니라 미리 정한 범위 안에서의 선택권이다. 좋은 에이전트는 무엇을 할 수 있는지뿐 아니라 무엇을 하면 안 되는지도 명확하다.

에이전트의 6 가지 구성요소

실무에서는 에이전트를 여섯 요소로 보면 단순해진다. 첫째 목표, 둘째 입력, 셋째 기억 또는 상태, 넷째 판단 규칙, 다섯째 도구, 여섯째 검증과 종료 조건이다. 예를 들어 시장조사 에이전트의 목표는 “후보 유통사 20 곳을 조사해 우선순위를 매긴다”이고, 입력은 국가와 제품군이다. 도구는 웹검색과 스프레드시트, 검증은 출처 링크와 중복 제거가 된다.

이 여섯 칸이 비어 있으면 AI가 똑똑해도 결과는 흔들린다. 특히 종료 조건이 중요하다. “충분히 조사해줘”보다 “신뢰 가능한 출처가 두 개 이상 확인된 회사만 최종 목록에 남겨라”가 훨씬 안정적이다. 에이전트는 목표를 향해 움직이는 시스템이므로, 어디서 멈출지까지 설계해야 한다.

2026년의 코딩 에이전트가 보여주는 변화

2026년의 코딩 에이전트는 코드 한 줄을 제안하는 도구를 넘어 파일을 읽고 수정하고, 테스트를 돌리고, 여러 작업을 병렬로 처리하는 방향으로 발전했다. OpenAI는 Codex를 소프트웨어 개발뿐 아니라 보고서, 스프레드시트, 프레젠테이션, 계약서 같은 지식 업무에도 활용되는 생산성 도구로 설명한다. 중요한 점은 특정 제품의 이름이 아니라 “AI가 산출물 자체를 다룬다”는 변화다.

비개발자에게 이 변화는 기회다. 과거에는 아이디어를 구현하려면 개발자에게 요구사항을 전달한 뒤 결과를 기다려야 했다. 지금은 작은 내부 도구와 프로토타입을 직접 만들어 본 뒤, 필요한 부분만 전문가에게 넘길 수 있다. 이는 개발자의 역할을 없애는 것이 아니라 문제정의와 검증의 속도를 높인다.

가장 먼저 버려야 할 오해

첫째, 에이전트는 사람처럼 모든 상황을 알아서 처리하지 않는다. 둘째, 자동화가 곧 무인화는 아니다. 셋째, 최신 모델을 쓰는 것만으로 업무 혁신이 생기지 않는다. 오히려 데이터가 영커 있거나 승인 규칙이 불명확하면 더 빠르게 잘못된 일을 반복할 수 있다.

따라서 시작점은 “어떤 AI를 살까”가 아니라 “어떤 업무를 어떤 품질 기준으로 반복하고 있는가”다. 업무가 정의되어 있지 않다면 AI는 그 혼란을 확대한다. 반대로 입력과 기준이 정리된 업무는 간단한 모델만으로도 큰 효과를 낼 수 있다.

실습 | 나의 첫 에이전트 후보 찾기

오늘 반복한 업무를 떠올리고 ① 입력이 일정한가 ② 판단 규칙을 말로 설명할 수 있는가 ③ 결과 형식이 정해져 있는가 ④ 사람이 최종 승인해도 되는가를 점검하라. 네 항목 중 세 개 이상이 “예”라면 첫 에이전트 후보로 적는다.

체크리스트

- ☐ 업무의 목표를 한 문장으로 말할 수 있다
- ☐ 입력 자료의 위치를 알고 있다
- ☐ 완료 여부를 판정할 기준이 있다
- ☐ 실패해도 되돌릴 수 있다

한 줄 정리

에이전트는 “똑똑한 챗봇”이 아니라 목표, 상태, 도구, 검증을 가진 업무 시스템이다.

CHAPTER 02

내 일을 AI 가 이해하게 만드는 법

AI 성능보다 먼저 바뀌어야 할 것은 내 머릿속의 암묵지를 밖으로 꺼내는 일이다

업무 설명이 모호하면 AI 도 모호해진다

사람은 조직의 맥락을 공유한다. “지난번 방식으로 정리해 주세요”라는 말도 동료에게는 통한다. AI 에게는 그렇지 않다. 지난번이 무엇인지, 어떤 기준을 적용했는지, 무엇을 빼야 하는지 알려주지 않으면 그럴듯한 추측으로 빈칸을 채운다. 그래서 AI 활용 능력은 프롬프트 기술보다 업무를 명시적으로 설명하는 능력에 가깝다.

업무를 설명할 때는 목적-입력-규칙-출력-검증 순서가 유용하다. “이 파일을 요약해 줘”보다 “임원 회의에서 5 분 안에 의사결정할 수 있도록, 매출 변동 원인과 위험요인만 추려 1 페이지 메모로 만들고 숫자는 원본 셀을 인용하라”가 훨씬 강하다.

암묵지를 규칙으로 바꾸는 인터뷰

오래 일한 사람은 왜 그렇게 판단하는지 설명하기 어렵다. “보면 알아요”라는 영역이 많다. 이럴 때 AI 를 인터뷰어로 쓰면 좋다. “내가 해외 바이어를 평가하는 기준을 추출하고 싶다. 한 번에 한 질문씩 물어봐라”라고 시키고 실제 경험을 답한다. AI 는 반복되는 기준을 찾아 체크리스트 초안으로 정리할 수 있다.

다만 추출된 규칙이 곧 진실은 아니다. 인터뷰 결과는 반드시 실제 사례에 적용해 보며 보정해야 한다. 좋은 업무 매뉴얼은 사람의 설명과 과거 데이터, 예외 사례가 함께 들어간다.

좋은 예시가 긴 설명보다 강하다

AI 에게 원하는 결과를 알려주는 가장 빠른 방법은 좋은 예시와 나쁜 예시를 함께 보여주는 것이다. 이메일이라면 “이 톤은 좋음 / 이 표현은 피함”, 보고서라면 “이 구조를 유지 / 이 항목은 삭제”처럼 명확히 대비시킨다. 이를 몇 개의 대표 사례로 정리하면 팀 전체가 같은 품질을 재현할 수 있다.

예시는 많을수록 좋은 것이 아니다. 서로 다른 상황을 대표하는 3~5 개 정도가 보통 충분하다. 너무 많은 예시를 넣으면 오히려 중요한 규칙이 묻힌다. 예시를 고를 때는 빈도가 높은 정상 사례, 위험한 예외 사례, 경계선 사례를 섞는다.

업무 사전 만들기

AI 프로젝트가 커질수록 용어가 중요해진다. “유효 바이어”, “재접촉”, “승인”, “고위험 문서” 같은 단어를 팀마다 다르게 이해하면 자동화가 흔들린다. 업무 사전은 용어, 정의, 예시, 반례, 책임자를 한 표에 모은 문서다.

업무 사전은 거창할 필요가 없다. 스프레드시트 한 장이면 된다. 중요한 것은 누구나 같은 단어를 같은 의미로 쓰게 만드는 것이다. 이 사전은 나중에 프롬프트, 데이터베이스 필드, 에이전트 규칙의 공통 기반이 된다.

실습 | 업무 설명서 1 장 만들기

반복 업무 하나를 골라 목적 / 입력 / 규칙 / 출력 / 검증 / 예외의 여섯 칸으로 정리하라. 각 칸은 최대 세 문장으로 쓴다. 마지막에 실제 결과물 한 개를 예시로 붙인다.

체크리스트

- ☐ 목적이 결과물 중심으로 쓰였다
- ☐ 입력 자료의 출처가 적혀 있다

- ☐ 판단 규칙이 최소 3 개 있다
- ☐ 예외 처리 방법이 있다

한 줄 정리

AI 에게 일을 잘 맡기는 첫 기술은 프롬프트가 아니라 내 일을 설명 가능한 구조로 바꾸는 것이다.

CHAPTER 03

좋은 프롬프트보다 좋은 업무 지시서

한 번 쓰고 버리는 질문이 아니라 반복 가능한 지시서를 만든다

프롬프트의 다섯 층

실무용 프롬프트는 역할, 목표, 자료, 제약, 출력 형식의 다섯 층으로 구성하면 안정적이다. 역할은 관점을 정하고, 목표는 성공 기준을 정한다. 자료는 근거를 제한하고, 제약은 하지 말아야 할 일을 알려준다. 출력 형식은 결과를 다음 업무에 바로 쓸 수 있게 한다.

예를 들어 “너는 수출 영업 매니저다”라는 역할만으로는 부족하다. “아래 기존 교신 이력과 가격표만 사용해, 재주문 가능성을 높이는 후속 메일을 180 단어 이내로 작성하라. 새로운 가격이나 납기는 추정하지 말고 확인 필요로 표시하라. 제목 3개와 본문 1개를 출력하라” 정도가 되어야 실제 업무 지시서가 된다.

체인 오브 프롬프트보다 단계 분리

복잡한 업무를 한 프롬프트에 길게 적으면 결과가 좋을 때도 있지만 오류를 찾기 어렵다. 실무에서는 조사, 분류, 작성, 검수처럼 단계를 분리하는 편이 안전하다. 각 단계의 입력과 출력이 명확하면 중간 검수가 가능하고, 특정 단계만 다시 실행할 수 있다.

예를 들어 제안서 자동화에서 “회사 조사부터 제안서까지 한 번에”보다 ① 후보 조사 ② 사실 검증 ③ 적합성 점수화 ④ 메시지 초안 ⑤ 사람 승인으로 나누면 근거 없는 정보가 최종 문서로 넘어갈 가능성을 줄일 수 있다.

구조화된 출력의 힘

AI 가 사람에게만 읽히는 문장을 내놓으면 다음 단계 자동화가 어렵다. 반대로 항목명이 정해진 표나 JSON 처럼 구조화된 형식으로 출력하면 스프레드시트나 API 가 바로 이어받을 수 있다. 에이전트는 문장 생성 능력보다 구조화된 판단 결과를 잘 만드는 것이 중요할 때가 많다.

다만 구조화 형식을 쓴다고 정확도가 자동으로 높아지는 것은 아니다. 필드별 정의와 허용값을 명시해야 한다. 예를 들어 상태값을 QUOTE, SAMPLE, FOLLOW_UP, HOLD 네 가지 중 하나로 제한하면 이후 워크플로가 단순해진다.

프롬프트도 버전 관리한다

업무 프롬프트는 문서가 아니라 소프트웨어에 가까운 자산이다. 한 문장을 바꿨는데 오류가 늘 수 있고, 특정 데이터에서는 잘 작동하지만 다른 데이터에서는 무너질 수 있다. 그래서 날짜와 버전, 변경 이유, 테스트 결과를 남기는 습관이 필요하다.

가장 간단한 방법은 파일명에 버전을 넣고 테스트 사례 5 개를 고정하는 것이다. 프롬프트를 수정할 때마다 같은 5 개 사례로 결과를 비교한다. 이 습관만으로도 “느낌상 좋아졌다”는 평가에서 벗어날 수 있다.

실습 | 재사용 프롬프트 템플릿 만들기

자주 하는 업무 하나를 골라 [역할] [목표] [입력] [판단 규칙] [금지사항] [출력형식] [검증] 칸을 채운다. 실제 사례 3 개로 테스트하고 버전 번호 v0.1 을 붙인다.

체크리스트

- ☐ 근거 없이 새 사실을 만들지 말라는 규칙이 있다
- ☐ 출력 형식이 다음 작업에 바로 쓸 수 있다

- ☐ 실패 사례가 최소 1 개 포함돼 있다
- ☐ 버전과 변경 이유를 기록한다

한 줄 정리

좋은 프롬프트는 멋진 문장이 아니라 재사용과 검증이 가능한 업무 명세다.

PART 2 일을 워크플로로 바꾸기

업무를 분해하고 컨텍스트와 승인 규칙을 설계해 안전한 자동화의 뼈대를 만든다.

CHAPTER 04

업무를 입력-판단-실행으로 쪼개라

자동화는 AI 에서 시작하지 않는다. 먼저 업무 흐름을 그려야 한다

업무의 세 종류

대부분의 사무 업무는 입력을 모으는 일, 판단하는 일, 실행하는 일로 나눌 수 있다. 이메일을 예로 들면 이메일을 읽고 필요한 정보를 추출하는 것이 입력, 문의 유형과 우선순위를 정하는 것이 판단, 답장 초안을 만들거나 담당자에게 전달하는 것이 실행이다. 이 세 가지가 뒤섞여 있을수록 자동화가 어렵다.

분해의 목적은 AI 를 많이 넣기 위한 것이 아니다. 오히려 AI 가 필요 없는 단계를 찾기 위한 것이다. 날짜 계산, 파일 이동, 정해진 양식 채우기처럼 규칙이 명확한 일은 일반 자동화가 더 안정적이다. AI 는 분류, 요약, 비교, 초안처럼 모호성이 있는 구간에 넣는 편이 좋다.

워크플로 다이어그램 한 장

복잡한 시스템을 시작할 때 가장 유용한 산출물은 화려한 기획서가 아니라 상자와 화살표 한 장이다. 각 상자에는 행동을 동사로 적는다. “메일 수신 → 발신자 확인 → 기존 거래 이력 조회 → 문의 유형 분류 → 답장 초안 → 사람 승인 → 발송 → 로그 저장”처럼 흐름을 만든다.

각 화살표에는 데이터가 무엇으로 이동하는지 적으면 더 좋다. 이메일 본문, 고객 ID, 가격표 버전, 승인자 같은 정보가 보이면 필요한 시스템과 권한이 드러난다. 이 다이어그램은 개발자와 비개발자가 같은 그림을 보게 만드는 공통 언어다.

병목을 찾는 네 질문

어떤 업무부터 자동화할지 결정할 때는 네 가지를 묻는다. 자주 반복되는가, 시간이 많이 드는가, 실수가 자주 나는가, 결과를 표준화할 수 있는가. 네 항목이 모두 높다면 우선순위가 높다. 반대로 자주 하지 않고 예외가 많으며 실패 비용이 큰 업무는 첫 프로젝트로 피하는 편이 낫다.

많은 조직이 눈에 띄는 생성 업무부터 자동화하려 한다. 하지만 실제 시간은 자료를 찾고 복사하고 형식을 맞추는 데 더 많이 쓰이는 경우가 있다. AI 프로젝트를 시작하기 전에 일주일만 작업 시간을 기록해보면 예상과 다른 병목이 보인다.

작은 단위로 성공을 정의한다

“영업을 자동화한다”는 프로젝트는 너무 크다. “과거 30 일간 후속 연락이 없는 바이어를 찾아 일일 목록을 만든다” 정도가 첫 단위로 좋다. 작은 성공은 측정하기 쉽고, 실패해도 원인을 찾기 쉽다.

작은 단위를 연결하면 큰 시스템이 된다. 연락 대상 탐지 → 바이어 정보 조회 → 메일 초안 → 승인 → 발송 → 반응 기록을 순서대로 붙이면 영업 에이전트가 된다. 에이전트 구축은 거대한 한 번의 개발보다 작은 검증 가능한 모듈을 이어 붙이는 과정이다.

실습 | 워크플로 지도 그리기

A4 한 장에 반복 업무를 시작부터 끝까지 상자로 그린다. 각 상자에 R(규칙자동화), A(AI), H(사람) 중 하나를 표시하고, 실패했을 때 되돌아갈 지점도 표시한다.

체크리스트

- ☐ AI 가 없어도 자동화 가능한 단계가 구분됐다
- ☐ 사람 승인이 필요한 단계가 표시됐다

- ☐ 각 단계의 입력과 출력이 보인다
- ☐ 실패 복구 지점이 있다

한 줄 정리

자동화의 품질은 모델의 지능보다 업무를 얼마나 잘 분해했는지에 좌우된다.

CHAPTER 05

컨텍스트 엔지니어링: AI에게 필요한 것만 보여주기

좋은 답은 더 많은 자료가 아니라 더 적절한 자료에서 나온다

컨텍스트는 AI의 작업대다

AI가 현재 작업을 위해 보고 있는 정보 전체를 컨텍스트라고 생각하면 이해가 쉽다. 지시문, 대화 기록, 첨부 문서, 검색 결과, 데이터베이스 조회 결과가 모두 작업대 위에 올라간다. 작업대가 너무 비어 있으면 추측이 늘고, 너무 어지러우면 중요한 정보를 놓친다.

컨텍스트 엔지니어링의 목표는 AI에게 모든 것을 주는 것이 아니라 지금 판단에 필요한 정보를 정확히 고르는 것이다. 계약서 검토에 전사 마케팅 자료까지 넣을 이유는 없다. 반대로 가격 협상 메일을 쓸 때 최신 가격표와 기존 조건이 빠지면 위험하다.

고정 정보와 변동 정보를 나눈다

업무에 쓰이는 정보는 크게 고정 정보와 변동 정보로 나눌 수 있다. 브랜드 톤, 승인 규칙, 제품 기본 스펙은 비교적 고정적이다. 재고, 환율, 최근 교신, 캠페인 일정은 계속 바뀐다. 두 종류를 섞어서 프롬프트에 하드코딩하면 금방 낡는다.

고정 정보는 지침 파일이나 지식베이스로, 변동 정보는 조회 가능한 데이터 소스로 관리하는 것이 좋다. 이 구분만 해도 “지난달 가격으로 메일을 보내는” 종류의 오류를 크게 줄일 수 있다.

검색과 기억은 다르다

AI 의 기억에만 의존하는 것과 필요할 때 원자료를 검색해 가져오는 것은 다르다. 업무에서는 최신성과 근거가 필요한 정보는 검색 또는 데이터 조회를 우선해야 한다. 특히 가격, 계약 조건, 규정, 일정처럼 틀리면 비용이 발생하는 정보는 기억보다 출처가 중요하다.

에이전트 설계에서 “무엇을 기억하게 할까”보다 “언제 무엇을 다시 조회하게 할까”가 더 중요한 경우가 많다. 기억은 편의를 주고, 검색은 신뢰를 준다. 둘을 목적에 따라 구분해야 한다.

컨텍스트 예산을 관리한다

자료를 계속 쌓으면 비용과 지연이 늘고 핵심이 묻힐 수 있다. 긴 프로젝트에서는 최근 작업 요약, 핵심 결정 목록, 참조 파일 인덱스처럼 컨텍스트를 압축하는 구조가 필요하다. 모든 과거 대화를 매번 넣는 방식은 확장성이 낮다.

실무적으로는 “항상 필요한 것”, “조건부로 필요한 것”, “요청 시 불러올 것” 세 층으로 나누면 좋다. 이 구조가 나중에 RAG 와 도구 호출 설계로 자연스럽게 이어진다.

실습 | 컨텍스트 다이어트

현재 반복 업무의 자료를 모두 적고, 각 자료 옆에 ALWAYS / WHEN NEEDED / NEVER 세 층으로 라벨을 붙인다. NEVER 에는 개인정보나 불필요한 기밀 자료도 포함한다.

체크리스트

- ☐ 최신성이 중요한 데이터가 표시됐다
- ☐ 항상 필요한 규칙과 필요할 때만 조회할 자료가 분리됐다
- ☐ 민감정보 입력 금지 기준이 있다

☐ 오래된 자료의 폐기 기준이 있다

한 줄 정리

AI 에게 많이 보여주는 것이 아니라, 지금 필요한 것을 정확히 보여주는 것이 컨텍스트 엔지니어링이다.

CHAPTER 06

사람의 승인을 설계하는 법

좋은 자동화는 사람을 없애지 않는다. 사람의 판단이 필요한 지점을 더 분명하게 만든다

Human Gate 가 필요한 이유

AI 는 빠르게 초안을 만들지만 책임을 질 수 없다. 가격 확정, 계약 조항, 규제 문구, 외부 발송처럼 실패 비용이 큰 행동은 사람이 승인하는 게 기본이다. 이를 Human Gate 라고 부를 수 있다. 핵심은 “사람이 마지막에 본다”가 아니라 어떤 조건에서 누구에게 승인을 요청할지 규칙으로 만드는 것이다.

예를 들어 메일 초안은 자동 생성하되 할인율이 10%를 넘거나 새로운 납기 약속이 포함되면 영업 책임자의 승인을 요구한다. 단순 감사 메일은 담당자가 한 번 확인하고 발송할 수 있다. 위험도에 따라 승인 강도를 다르게 해야 병목이 생기지 않는다.

위험을 세 단계로 나눈다

실무에서는 LOW, MEDIUM, HIGH 처럼 단순한 등급이 유용하다. LOW 는 내부 요약이나 공개자료 정리, MEDIUM 은 고객 대상 초안이나 내부 의사결정 자료, HIGH 는 계약·가격·규제·개인정보·대외 공표로 볼 수 있다. 등급마다 자동 실행 범위를 다르게 설정한다.

위험 등급은 기술팀만 정해서는 안 된다. 실제 업무 책임자가 오류 비용을 가장 잘 안다. AI 거버넌스는 문서가 아니라 “이 행동은 자동으로 해도 되는가”를 조직이 합의하는 과정이다.

승인 화면은 짧아야 한다

자동화가 실패하는 흔한 이유 중 하나는 승인 과정이 너무 번거로운 것이다. 사람이 원문과 AI 결과를 비교하기 위해 여러 시스템을 오가야 한다면 결국 자동화를 쓰지 않게 된다. 승인 화면에는 판단에 필요한 근거를 함께 보여줘야 한다.

좋은 승인 화면은 “AI가 무엇을 했는지, 어떤 근거를 썼는지, 무엇이 바뀌었는지, 위험 신호가 무엇인지”를 한눈에 보여준다. 승인자의 역할은 처음부터 다시 일하는 것이 아니라 위험한 부분만 확인하는 것이다.

로그는 책임과 학습의 자산

누가 언제 무엇을 승인했는지, 어떤 버전의 데이터와 프롬프트가 쓰였는지 기록하면 문제가 생겼을 때 원인을 찾을 수 있다. 로그는 감사를 위한 부담이 아니라 성능 개선을 위한 학습 데이터다.

같은 유형의 수정이 반복된다면 프롬프트나 규칙을 개선할 수 있다. 사람이 매번 “이 표현은 과장”이라고 고친다면 그 수정은 다음 버전의 기준이 되어야 한다. Human Gate는 마지막 방어선인 동시에 시스템이 배우는 센서다.

실습 | 승인 매트릭스 만들기

업무 행동 10 개를 적고 LOW/MEDIUM/HIGH 로 분류한다. 각 등급별로 자동 실행 / 담당자 승인 / 책임자 승인 중 하나를 정하고, 승인 시 반드시 확인할 근거를 적는다.

체크리스트

- ☐ 외부 발송에는 승인 규칙이 있다
- ☐ 가격·계약·규정 문구는 고위험으로 분류됐다
- ☐ 승인자가 한 화면에서 근거를 볼 수 있다

☐ 승인·수정 이력이 남는다

한 줄 정리

Human-in-the-loop 는 자동화의 타협이 아니라 신뢰할 수 있는 자동화를 만드는 핵심 설계다.

PART 3 코딩을 몰라도 만드는 시대

자연어로 소프트웨어를 만들고 버전 관리하는 실전 감각을 익힌다.

CHAPTER 07

바이브 코딩의 원리

코드를 직접 쓰는 능력보다 요구사항을 명확히 하고 결과를 검증하는 능력이 중요해졌다

바이브 코딩은 대충 말하는 코딩이 아니다

바이브 코딩은 자연어로 원하는 기능을 설명하고 AI 가 코드를 작성·수정하도록 하면서 사람이 방향과 결과를 관리하는 방식이다. “감으로 코딩한다”는 표현 때문에 즉흥적으로 보이지만, 실무에서는 요구사항과 테스트가 분명할수록 훨씬 강력하다.

좋은 바이브 코딩은 제품 관리자처럼 일하는 것이다. 누가 쓰는지, 어떤 문제를 해결하는지, 성공 조건은 무엇인지, 화면과 데이터가 어떻게 흘러가는지 설명한 뒤 AI 가 구현하도록 맡긴다. 코드는 AI 가 많이 쓰지만 책임은 여전히 사람에게 있다.

첫 앱은 내부 도구가 좋다

처음부터 고객용 서비스나 결제 시스템을 만들기보다 개인 또는 팀 내부에서 쓰는 작은 도구로 시작하는 것이 좋다. 파일 이름 일괄 변경, 견적서 생성, 바이어 목록 필터링, 간단한 대시보드처럼 실패 비용이 낮고 효과가 눈에 보이는 과제가 적합하다.

내부 도구는 사용자 인터뷰도 쉽다. 바로 옆 동료가 불편한 점을 말해주고, 다음 버전에서 즉시 수정할 수 있다. 이 빠른 반복 경험이 바이브 코딩의 핵심 감각을 만든다.

요구사항은 화면보다 행동으로 쓴다

“예쁜 대시보드를 만들어줘”보다 “사용자가 국가를 선택하면 해당 국가의 바이어만 보이고, 최근 접촉일 순으로 정렬되며, CSV 로 내려받을 수 있어야 한다”가 좋은 요구사항이다. 기능은 사용자의 행동과 시스템 반응으로 작성한다.

화면 디자인은 그다음이다. 기능 목록, 데이터 필드, 예외 상황, 완료 기준이 먼저 있어야 AI 가 코드 구조를 안정적으로 잡는다. 디자인만 먼저 요구하면 겉은 그럴듯하지만 실제 업무에 쓸 수 없는 결과가 나오기 쉽다.

테스트를 요구사항과 함께 쓴다

기능을 설명할 때 “성공했는지 어떻게 확인할지”도 함께 적는다. 예를 들어 “빈 파일을 업로드하면 오류 메시지가 보여야 한다”, “날짜가 없는 행은 분석에서 제외하고 개수를 표시해야 한다” 같은 조건이다.

테스트는 개발자만의 일이 아니다. 비개발자는 업무 규칙을 알고 있으므로 어떤 결과가 맞는지 가장 잘 정의할 수 있다. AI 에게 코드를 맡길수록 테스트 시나리오는 더 중요해진다.

실습 | 30 분짜리 내부 도구 기획

불편한 반복 업무 하나를 골라 사용자 / 문제 / 입력 / 핵심 기능 3 개 / 실패 시 메시지 / 성공 기준을 한 페이지에 적는다. 이 문서만으로 AI 에게 첫 프로토타입을 요청한다.

체크리스트

- ☐ 기능이 3~5 개로 제한돼 있다
- ☐ 데이터 입력 형식이 적혀 있다
- ☐ 오류 상황이 정의돼 있다

☐ 성공 테스트가 있다

한 줄 정리

바이브 코딩은 코드를 모르는 사람의 지름길이 아니라 문제를 정확히 말하는 사람의 레버리지다.

CHAPTER 08

Codex 와 코딩 에이전트로 첫 앱 만들기

설치보다 중요한 것은 작업 단위를 작게 만들고 에이전트에게 확인 가능한 산출물을 요구하는 것이다

코딩 에이전트는 파일과 프로젝트를 다룬다

코딩 에이전트는 대화창에 코드 조각을 주는 수준을 넘어 프로젝트 폴더의 파일을 읽고, 여러 파일을 수정하고, 명령을 실행하고, 테스트 결과를 확인하는 흐름으로 작동한다. 따라서 사람은 “어느 파일을 고칠지”보다 “어떤 동작이 완성되어야 하는지”를 설명하는 데 집중할 수 있다.

작업을 맡길 때는 프로젝트 목적과 현재 상태, 수정 범위, 금지 사항, 완료 기준을 짧게 제공한다. 기존 기능을 망가뜨리지 말라는 조건과 변경 후 테스트를 실행하라는 지시를 기본 습관으로 넣는다.

한 번에 한 기능

첫 프로젝트에서 가장 흔한 실수는 로그인, 결제, 관리자, 검색, AI 추천을 한 번에 요구하는 것이다. 에이전트가 많은 일을 할 수 있어도 사람이 결과를 검증할 수 없으면 프로젝트는 금방 흐트러진다.

기능을 세로로 얇게 자른다. 예를 들어 “CSV 업로드 → 표 표시”를 먼저 끝낸 뒤 “필터”, 그다음 “AI 요약”, 마지막에 “저장”을 붙인다. 매 단계마다 작동하는 버전을 남기면 되돌아갈 수 있다.

에러 메시지를 그대로 보여줘라

코딩 지식이 부족할수록 오류를 해석하려고 애쓰기보다 원문을 그대로 에이전트에게 보여주는 게 좋다. “안 돼”보다 오류 로그, 재현 단계, 기대 동작을 함께 주면 훨씬 빠르게 해결된다.

다만 AI 가 같은 수정만 반복한다면 작업을 멈추고 원인을 가설별로 나누게 한다. 환경 문제, 데이터 문제, 코드 문제를 구분하면 무한 수정 루프를 피할 수 있다.

작동과 안전은 별개다

프로토타입이 실행된다고 바로 배포하면 안 된다. 인증, 권한, 비밀키, 개인정보, 입력 검증, 로그 등은 별도의 검토가 필요하다. 특히 API 키를 코드에 직접 적거나 공개 저장소에 올리는 실수는 피해야 한다.

첫 앱의 목적은 “완벽한 제품”이 아니라 문제 해결 가능성을 증명하는 것이다. 외부 서비스로 배포하기 전에는 전문가 검토 또는 보안 체크리스트를 거치는 것이 안전하다.

실습 | 첫 기능 구현 지시서

코딩 에이전트에게 “현재 프로젝트를 먼저 읽고 변경 계획을 5 줄 이내로 제시한 뒤, 승인 없이 대규모 구조 변경은 하지 말 것. 기능 구현 후 테스트 결과와 바뀐 파일 목록을 보고할 것”이라는 작업 규칙을 붙인다.

체크리스트

- ☐ 작업 범위가 한 기능이다
- ☐ 변경 전 계획을 확인한다
- ☐ 변경 후 테스트를 실행한다
- ☐ 비밀키를 코드에 넣지 않는다

한 줄 정리

코딩 에이전트의 강점은 속도다. 그 속도를 통제하는 방법은 작은 작업, 테스트, 버전 관리다.

CHAPTER 09

GitHub 로 AI 작업을 관리하기

AI 가 만든 파일이 늘수록 “어제 잘 되던 버전”으로 돌아갈 수 있는 능력이 중요해진다

GitHub 는 백업 이상의 도구

GitHub 를 처음 보면 개발자용 서비스처럼 보이지만, 비개발자에게 가장 중요한 가치는 변경 이력을 남기는 데 있다. 어떤 파일이 언제 어떻게 바뀌었는지, 어느 버전이 정상인지 알 수 있다. AI 가 빠르게 코드를 수정하는 시대에는 이 기능이 특히 중요하다.

저장소를 프로젝트 하나의 작업실로 생각하면 된다. README 에는 목적과 사용법, docs 폴더에는 요구사항과 결정 기록, issues 에는 할 일과 오류를 남긴다. 이 구조만 지켜도 프로젝트의 맥락이 보존된다.

커밋은 작은 스냅샷

커밋은 “이 시점의 상태를 저장한다”는 의미다. 기능 하나가 정상 동작할 때마다 커밋하면 다음 수정이 실패해도 돌아갈 수 있다. “수정” 같은 메시지보다 “CSV 업로드 시 날짜 검증 추가”처럼 무엇이 바뀌었는지 적는 습관이 좋다.

AI 에게도 커밋 단위를 지키게 할 수 있다. 기능 구현 후 변경 내용을 요약하고, 사용자가 확인한 뒤 커밋하도록 하면 자동화와 통제가 균형을 이룬다.

이슈는 AI 에게 줄 업무 티켓

이슈에 목적, 현재 문제, 완료 기준을 적어두면 그대로 에이전트의 작업 지시서가 된다. 여러 사람이 협업할 때도 “무엇을 왜 바꾸는지”가 보인다.

좋은 이슈는 길 필요가 없다. 배경 3 줄, 해야 할 일 3 개, 완료 조건 3 개 정도면 충분하다. 핵심은 한 이슈에 한 결과를 두는 것이다.

프로젝트 기록이 조직 지식이 된다

AI 프로젝트는 성공한 코드보다 실패한 이유가 더 큰 자산일 때가 있다. 어떤 프롬프트가 안 됐는지, 어떤 데이터가 문제였는지, 왜 특정 구조를 선택했는지 기록하면 다음 프로젝트에서 같은 실수를 줄인다.

GitHub 의 이슈와 README 를 단순 개발 기록이 아니라 의사결정 로그로 쓰면, 사람이 바뀌어도 프로젝트 맥락이 남는다. 이것이 개인의 AI 활용을 조직 역량으로 바꾸는 첫 단계다.

실습 | 저장소 기본 구조 만들기

프로젝트 저장소에 README.md, docs/requirements.md, docs/decisions.md, data/README.md 를 만든다. 개인정보나 비밀키는 저장하지 않는다는 규칙을 README 첫 부분에 명시한다.

체크리스트

- ☐ 정상 작동 버전이 커밋되어 있다
- ☐ 요구사항 문서가 있다
- ☐ 결정 이유가 기록된다
- ☐ 비밀정보 제외 규칙이 있다

한 줄 정리

버전 관리는 개발자의 습관이 아니라 AI 에게 일을 맡기는 모든 사람의 안전장치다.

PART 4 회사 자료와 데이터를 연결하기

회사 문서, 데이터, 외부 API 를 AI 와 연결해 근거 있는 실행을 가능하게 한다.

CHAPTER 10

RAG: 우리 회사 문서를 읽는 AI

AI 가 모르는 회사 지식을 답하게 하려면 기억시키는 것이 아니라 필요할 때 찾아오게 해야 한다

RAG 를 가장 쉽게 이해하기

RAG 는 질문이 들어왔을 때 관련 문서를 검색해 필요한 부분을 가져오고, 그 내용을 근거로 답하게 하는 방식이다. 사람으로 치면 시험 전에 모든 책을 외우는 대신 질문을 받고 책에서 관련 페이지를 찾아 답하는 것과 비슷하다.

회사 규정, 제품 사양, FAQ, 매뉴얼처럼 근거 문서가 있는 업무에 특히 유용하다. 모델 자체를 다시 학습시키지 않아도 문서가 바뀌면 인덱스를 갱신해 최신 내용을 반영할 수 있다.

문서를 넣기 전에 정리한다

RAG 품질은 모델보다 문서 품질에 크게 좌우된다. 스캔이 엉망이거나 표가 깨져 있고 같은 문서의 구버전이 여러 개 섞여 있으면 검색 결과도 흔들린다. 먼저 문서명, 버전, 날짜, 소유자, 유효 여부를 정리해야 한다.

특히 규정과 가격표는 “최신 승인본” 표시가 중요하다. AI 가 오래된 문서를 찾지 않도록 폐기 정책과 버전 메타데이터를 넣는다.

청크와 메타데이터

긴 문서를 통째로 검색하기보다 의미 단위로 나누어 저장하는데 이를 청크라고 한다. 너무 작으면 맥락이 끊기고, 너무 크면 관련 없는 내용이 같이 따라온다. 문서 구조에 맞춰 제목과 표 단위로 나누는 것이 기본이다.

메타데이터는 검색의 필터다. 국가, 제품, 문서종류, 발행일, 버전 같은 정보를 붙이면 “일본용 최신 라벨 규정만”처럼 범위를 좁힐 수 있다.

답보다 출처를 먼저 본다

RAG 시스템에서 가장 중요한 UI 는 화려한 채팅창이 아니라 출처 표시일 수 있다. 답변 옆에 문서명과 페이지, 링크를 보여주면 사용자가 근거를 확인할 수 있다.

답이 그럴듯해도 출처가 없다면 고위험 업무에는 쓰기 어렵다. “근거를 찾지 못하면 모른다고 답하라”는 규칙을 기본값으로 두는 것이 좋다.

실습 | 문서 RAG 준비 목록

회사 문서 20 개만 골라 문서명 / 문서종류 / 버전 / 발행일 / 소유자 / 최신여부를 표로 만든다. 중복과 구버전을 제거한 뒤에야 RAG 구축을 시작한다.

체크리스트

- ☐ 최신본과 구버전이 구분된다
- ☐ 문서마다 소유자가 있다
- ☐ 검색 필터에 쓸 메타데이터가 있다
- ☐ 답변에 출처를 표시한다

한 줄 정리

RAG 는 AI 에게 지식을 “외우게” 하는 기술이 아니라 필요한 근거를 “찾아오게” 하는 설계다.

CHAPTER 11

스프레드시트와 데이터를 분석하는 AI

AI 가 숫자를 읽는 것과 좋은 분석을 하는 것은 다르다. 질문과 데이터 정의가 먼저다

분석은 열 이름에서 시작한다

엑셀이나 CSV 를 AI 에 넘기기 전에 각 열이 무엇을 의미하는지 확인해야 한다. 주문일과 결제일, 판매가와 정산가, 수량과 건수처럼 비슷해 보이는 필드가 다르면 결론도 달라진다. 데이터 사전 없이 시작하면 AI 가 추측한다.

분석 전에 “언제 / 무엇이 / 얼마에 / 몇 개”처럼 핵심 질문에 필요한 열을 정리하고, 자료에 없는 값은 없는 그대로 표시한다. 없는 데이터를 AI 가 만들어 채우게 하지 않는다.

정제와 분석을 분리한다

공백, 중복, 날짜 형식, 숫자와 문자열 혼용 같은 문제를 먼저 정리한 뒤 분석한다. 정제가 끝나지 않은 상태에서 AI 에게 매출 원인을 묻는 것은 흐린 렌즈로 사진을 보는 것과 같다.

정제 단계의 변경 내역도 남긴다. 어떤 행을 삭제했고 어떤 값을 치환했는지 기록해야 나중에 분석 결과를 재현할 수 있다.

AI 가 잘하는 분석과 못하는 분석

AI 는 데이터 요약, 그룹화, 패턴 설명, 시각화 코드 생성, 가설 제안에 강하다. 하지만 표본이 작거나 인과관계가 필요한 문제에서 확신에 찬 설명을 할 수 있다. 따라서 “무슨 일이 일어났나”와 “왜 일어났나”를 구분해야 한다.

매출이 떨어진 것과 광고 중단이 동시에 일어났다고 해서 광고가 원인이라고 단정할 수는 없다. AI 에게 가능한 설명과 필요한 추가 데이터를 분리해 달라고 요구한다.

경영 질문으로 변환한다

좋은 분석은 그래프에서 끝나지 않는다. “상위 20% 상품이 매출의 몇 %를 만드는가”, “재구매 주기가 늘어난 고객군은 누구인가”, “할인율 증가가 수량 증가로 이어졌는가”처럼 의사결정에 연결되는 질문을 만든다.

분석 보고서의 마지막에는 항상 “다음 행동”과 “추가 확인할 데이터”를 넣는다. AI 가 발견한 패턴을 실제 실험으로 연결해야 데이터가 가치가 된다.

실습 | 데이터 사전 만들기

최근 자주 쓰는 스프레드시트 하나를 골라 열 이름 / 정의 / 단위 / 예시 / 결측 처리 / 민감정보 여부를 표로 만든다. AI 분석은 그 표와 함께 시작한다.

체크리스트

- ☐ 열 정의가 있다
- ☐ 개인정보 열이 식별됐다
- ☐ 결측치 처리 규칙이 있다
- ☐ 분석 질문이 의사결정과 연결된다

한 줄 정리

AI 데이터 분석의 품질은 화려한 모델보다 데이터 정의와 질문의 품질에서 시작한다.

CHAPTER 12

API 와 함수 호출: AI 에게 손과 발 달 기

모델이 외부 시스템을 직접 아는 것이 아니라, 필요한 도구를 선택하고 호출 결과를 받아 쓰게 만든다

API 는 시스템 사이의 주문서

API 를 어렵게 생각할 필요가 없다. 한 프로그램이 다른 프로그램에게 정해진 형식으로 요청하고 결과를 받는 통로다. 날씨 조회, 환율, CRM 검색, 이메일 발송처럼 외부 기능을 AI 와 연결할 때 API 가 사용된다.

AI 는 “어떤 기능을 언제 쓸지”를 판단할 수 있지만 실제 호출은 시스템이 수행한다. 이 경계를 이해하면 보안과 오류 처리도 명확해진다.

함수 호출의 기본 흐름

함수 호출은 모델에게 사용할 수 있는 함수의 이름과 입력 형식을 알려주고, 사용자의 요청에 맞는 함수를 선택해 구조화된 인자를 반환하게 하는 방식이다. 이후 애플리케이션이 실제 API 를 호출하고 결과를 다시 모델에게 전달한다.

이 구조의 장점은 모델이 임의로 외부 시스템을 만지는 것이 아니라 허용된 함수만 쓴다는 점이다. 도구 목록 자체가 권한의 경계가 된다.

도구는 작고 명확하게

“모든 CRM 작업” 같은 거대한 함수보다 `get_customer`, `create_draft`, `update_status` 처럼 작은 기능이 안전하다. 입력값 검증과 실패 메시지도 설계하기 쉽다.

특히 쓰기 작업은 읽기 작업보다 위험하다. 조회 함수와 수정 함수를 분리하고, 수정·발송에는 사람 승인을 넣는 것이 좋다.

실패를 정상 흐름으로 본다

API 는 실패한다. 네트워크가 끊기고, 권한이 만료되고, 예상과 다른 데이터가 온다. 에이전트가 실패를 숨기고 추측해서 계속 진행하면 더 큰 문제가 된다.

따라서 함수에는 성공/실패 상태와 오류 메시지를 명확히 반환하고, 실패 시 재시도 횟수와 중단 규칙을 정한다. “모르면 멈춘다”는 설계가 중요하다.

실습 | 도구 카탈로그 작성

에이전트에게 연결하고 싶은 외부 기능 10 개를 적고 READ / WRITE / SEND 세 종류로 분류한다. WRITE 와 SEND 에는 승인 필요 여부를 표시한다.

체크리스트

- ☐ 읽기와 쓰기 권한이 분리됐다
- ☐ 함수 입력값이 명확하다
- ☐ 실패 시 중단 규칙이 있다
- ☐ 발송은 승인 후 수행한다

한 줄 정리

API 는 AI 의 손과 발이지만, 어떤 손발을 줄지 결정하는 것은 시스템 설계자의 책임이다.

PART 5 에이전트를 실제 업무에 배치하기

리서치, 영업, 콘텐츠, 멀티 에이전트에 적용해 실제 업무 단위의 시스템을 만든다.

CHAPTER 13

리서치 에이전트

검색을 많이 하는 것이 아니라 근거를 남기고 비교 가능한 형태로 만드는 것이 목표다

검색과 리서치는 다르다

검색은 정보를 찾는 행동이고 리서치는 질문에 답하기 위해 근거를 수집·평가·정리하는 과정이다. 에이전트에게 “조사해줘”라고만 하면 검색 결과를 요약하는 데 그칠 수 있다.

리서치 지시에는 질문, 포함 기준, 제외 기준, 출처 우선순위, 날짜 범위, 출력 표를 함께 정의해야 한다.

출처 계층을 만든다

공식 기관, 회사 공식 발표, 원문 보고서, 신뢰할 수 있는 언론, 커뮤니티 의견은 같은 무게가 아니다. 사실 확인에는 1차 자료를 우선하고, 시장 반응을 볼 때는 커뮤니티 자료를 보조적으로 쓴다.

에이전트가 각 주장 옆에 출처 링크와 발행일을 남기게 하면 나중에 업데이트도 쉬워진다.

비교 표준을 먼저 정한다

후보 유통사를 조사한다면 회사마다 다른 이야기를 길게 쓰는 것보다 점포 수, 카테고리, 지역, 한국제품 취급 이력, 담당 부서, 근거 링크 같은 고정 필드를 먼저 정한다.

같은 형식으로 데이터를 모아야 우선순위 점수화와 후속 자동화가 가능하다. 리서치는 글쓰기보다 데이터 수집 설계에 가깝다.

최종 산출물은 결론과 불확실성을 함께

좋은 리서치 보고서는 “무엇을 알았는가”뿐 아니라 “무엇을 아직 모르는가”를 보여준다. 확인되지 않은 점, 서로 충돌하는 출처, 추가 연락이 필요한 항목을 따로 표시한다.

에이전트가 확신 점수를 임의로 만드는 것보다 근거 수, 출처 종류, 최신성 같은 관찰 가능한 기준으로 신뢰도를 표현하는 편이 낫다.

실습 | 시장조사 템플릿

한 국가와 한 제품을 정해 후보 10 곳을 조사한다. 필드는 회사명 / 채널 / 점포수 / 취급카테고리 / 한국제품 이력 / 최근 뉴스 / 연락처 / 근거 링크 / 다음 행동으로 고정한다.

체크리스트

- ☐ 출처 우선순위가 정의됐다
- ☐ 모든 사실에 링크가 있다
- ☐ 확인되지 않은 항목이 따로 표시됐다
- ☐ 다음 행동이 정해졌다

한 줄 정리

리서치 에이전트의 성과는 정보량이 아니라 검증 가능한 비교표와 다음 행동이다.

CHAPTER 14

영업·고객관리 에이전트

영업 자동화의 핵심은 “잘 쓰는 메일”보다 고객 맥락을 놓치지 않는 것이다

CRM 의 문제는 입력이 아니라 기억의 분산

중소 조직의 영업 정보는 이메일, 메신저, 명함, 엑셀, 담당자 기억에 흩어져 있다. 이 상태에서 AI 에게 메일을 잘 써 달라고 해도 고객의 과거 맥락을 놓칠 수 있다.

먼저 회사, 담당자, 관심제품, 마지막 접촉일, 단계, 다음 행동 같은 핵심 필드를 통합해야 한다. AI 에이전트는 그 위에서 작동한다.

문의 유형을 구조화한다

견적, 샘플, 후속, 클레임, 보류 등 자주 반복되는 문의 유형을 정의하면 에이전트가 다음 행동을 고르기 쉬워진다. 유형별 필요한 데이터와 승인 규칙도 다르게 설정한다.

예를 들어 견적 문의는 최신 가격표가 필요하고, 샘플 문의는 배송지와 수량 확인이 필요하다. 분류 자체보다 분류 후 어떤 체크리스트를 실행할지가 중요하다.

초안 자동화와 약속 자동화는 다르다

AI 가 답장 초안을 만드는 것은 비교적 낮은 위험이지만 가격, 납기, 독점권처럼 회사의 의무를 만드는 문장을 자동 확정하면 위험하다. 초안 단계에서는 불확실한 값을 “확인 필요”로 표시하게 한다.

사람이 승인한 정보만 외부에 발송되도록 하면 속도와 책임을 함께 확보할 수 있다.

후속 연락이 진짜 자동화 포인트

영업에서 큰 손실은 완벽하지 않은 첫 메일보다 후속 연락을 잇는 데서 생길 수 있다. 마지막 접촉일과 거래 단계를 기준으로 오늘 연락할 대상을 자동 제안하는 기능은 구현이 단순하면서 효과가 크다.

반응 결과를 다시 CRM에 기록하면 어떤 메시지와 시점이 효과적이었는지 학습할 수 있다.

실습 | 후속관리 규칙 만들기

영업 단계별로 “몇 일 반응이 없으면 어떤 행동을 할지”를 표로 만든다. 예: SAMPLE_SENT 7일 → 배송 확인, QUOTE_SENT 5일 → 검토 여부 확인.

체크리스트

- ☐ 고객 정보가 한 곳에 있다
- ☐ 문의 유형과 다음 행동이 연결돼 있다
- ☐ 가격·납기는 사람 승인이다
- ☐ 후속 기준일이 정의됐다

한 줄 정리

영업 에이전트는 메일 작성기가 아니라 고객 맥락과 다음 행동을 놓치지 않는 운영 시스템이다.

CHAPTER 15

콘텐츠 · 보고서 에이전트

생성 속도보다 브랜드 사실과 근거를 일관되게 유지하는 구조가 먼저다

콘텐츠 자동화의 함정

AI 는 글과 이미지를 빠르게 만든다. 문제는 빨라질수록 잘못된 사실과 과장 표현도 더 많이 퍼질 수 있다는 점이다. 브랜드 콘텐츠 자동화에는 사실 원본과 표현 규칙이 필요하다.

제품명, 용량, 핵심 특징, 인증, 금지 표현을 Brand Facts Master 같은 단일 원본으로 관리하면 여러 채널에서 일관성을 높일 수 있다.

보고서는 데이터에서 시작한다

주간 보고서를 자동화할 때는 문장 생성보다 숫자 수집과 정의가 먼저다. 같은 매출이라도 주문 기준인지 정산 기준인지 다르면 결론이 달라진다.

원자료 → 계산 → 차트 → 해석 → 행동 제안 순서를 분리하면 숫자 오류를 찾기 쉽다. AI 가 숫자를 직접 계산하기보다 코드나 스프레드시트 수식을 사용하게 하는 것도 방법이다.

콘텐츠 파이프라인을 표준화한다

기획, 초안, 사실 검증, 톤 교정, 시각자료, 승인, 게시, 성과 기록을 단계로 나누면 팀이 같은 품질을 유지할 수 있다.

AI 는 초안과 변형 작업에 강하다. 최종 주장과 브랜드 책임은 사람이 확인한다.

콘텐츠도 재사용 가능한 자산으로

한 번 만든 좋은 글을 카드뉴스, 이메일, FAQ, 영업자료로 변환하면 생산성이 크게 높아진다. 이때 핵심 사실은 고정하고 형식과 톤만 바꾸는 것이 안전하다.

어떤 콘텐츠가 성과를 냈는지 기록해 다음 기획에 반영하면 생성형 AI 가 단순 제작 도구에서 학습 시스템으로 바뀐다.

실습 | 콘텐츠 원본 사전 만들기

제품 또는 서비스 하나에 대해 사실 20 개, 금지 표현 10 개, 톤 원칙 5 개를 정리한다. 이후 모든 콘텐츠 프롬프트가 이 원본을 참조하게 한다.

체크리스트

- ☐ 사실 원본이 하나다
- ☐ 숫자의 출처가 있다
- ☐ 금지 표현이 정의됐다
- ☐ 게시 전 승인 단계가 있다

한 줄 정리

콘텐츠 자동화의 핵심은 많이 만드는 것이 아니라 사실과 톤을 흔들리지 않게 재사용하는 것이다.

CHAPTER 16

멀티 에이전트: AI 팀을 운영하는 법

하나의 슈퍼 에이전트보다 역할이 다른 작은 에이전트가 협업하는 구조가 관리하기 쉽다

역할 분리가 필요한 순간

한 에이전트에게 검색, 분석, 작성, 검수를 모두 맡기면 빠르지만 오류 원인을 찾기 어렵다. 업무가 복잡해질수록 역할을 나눈다. 조사 에이전트는 근거 수집, 분석 에이전트는 비교, 작성 에이전트는 문서화, 검수 에이전트는 규칙 위반을 찾게 할 수 있다.

역할 분리는 사람 조직과 비슷하지만 더 엄격한 입출력 계약이 필요하다. 각 에이전트가 무엇을 받아 무엇을 내놓는지 정해져야 한다.

오케스트레이터의 역할

멀티 에이전트 구조에서 중앙 조정자는 작업을 나누고 순서를 정하며 실패를 처리한다. 모든 내용을 직접 생성하기보다 어떤 전문 에이전트에게 넘길지 판단한다.

오케스트레이터가 너무 많은 권한을 가지면 복잡성이 커진다. 처음에는 단순한 순차 흐름부터 시작하고, 정말 필요한 곳에서만 병렬 작업과 재시도를 넣는다.

검수 에이전트를 독립시켜라

작성한 에이전트가 스스로 검수하면 같은 편향을 반복할 수 있다. 가능하면 별도의 검수 단계에 다른 지시와 기준을 사용한다.

예를 들어 보고서 검수 에이전트는 문체보다 숫자 출처, 날짜, 주장 근거, 누락 필드를 검사하게 한다.

팀이 커질수록 로그가 중요하다

여러 에이전트가 협업하면 어느 단계에서 잘못됐는지 추적해야 한다. 각 단계의 입력, 출력, 사용 도구, 시간, 오류를 기록한다.

멀티 에이전트는 화려하지만 운영 비용도 증가한다. 한 에이전트로 충분한 문제는 굳이 나누지 않는 것이 좋은 설계다.

실습 | 3인 AI 팀 설계

현재 업무 하나를 조사자 / 작성자 / 검수자 3개 역할로 나누고 각 역할의 입력, 출력, 완료 기준을 한 줄씩 적는다. 마지막에는 사람이 승인한다.

체크리스트

- ☐ 역할 간 입출력이 명확하다
- ☐ 검수 역할이 독립돼 있다
- ☐ 실패한 단계만 다시 실행할 수 있다
- ☐ 최종 승인자가 있다

한 줄 정리

멀티 에이전트의 목적은 AI를 많이 쓰는 것이 아니라 책임과 검증 단위를 분리하는 것이다.

PART 6 연결과 자동화의 완성

MCP와 자동화, 거버넌스를 통해 연결된 AI를 운영 가능한 체계로 만든다.

CHAPTER 17

MCP: AI 와 도구를 표준으로 연결하기

각 도구마다 제각각 연결하던 세계에서, AI 가 사용할 리소스와 기능을 표준 방식으로 노출하는 흐름이 커지고 있다

MCP 를 비개발자 관점에서 이해하기

Model Context Protocol 은 AI 애플리케이션이 외부의 자료와 도구를 일관된 방식으로 발견하고 사용할 수 있게 하는 프로토콜이다. 쉽게 말해 각 서비스와 AI 사이에 공통 규격의 어댑터를 두는 생각에 가깝다.

MCP 서버는 리소스, 프롬프트, 도구 같은 기능을 제공하고, AI 애플리케이션은 이를 연결해 사용한다. 규격이 통일되면 같은 연결을 여러 AI 클라이언트에서 재사용하기 쉬워진다.

API 와 MCP 의 관계

API 가 없어지는 것은 아니다. MCP 서버 내부에서 기존 API 를 호출할 수도 있다. 차이는 AI 관점에서 도구를 발견하고 호출하는 인터페이스를 표준화한다는 데 있다.

조직 입장에서는 권한과 감사 정책을 중앙화할 가능성이 커진다. 어떤 도구를 어떤 사용자에게 보여줄지 통제할 수 있다.

2026 년 MCP 의 확장

MCP 는 빠르게 진화하고 있으며 2026 년 사양은 상태 관리, 인증, 확장 구조, 장기 작업 같은 운영 요구를 강화하는 방향으로 발전했다. 따라서 구현 시에는 최신 공식 사양을 확인해야 한다.

책에서는 세부 프로토콜보다 설계 원칙이 중요하다. 리소스는 읽기 자료, 도구는 행동, 권한은 경계라는 관점으로 이해하면 충분하다.

연결이 쉬워질수록 통제가 중요하다

AI 가 여러 도구를 쉽게 쓸 수 있게 되면 잘못된 도구 선택의 위험도 커진다. 특히 메일 발송, 파일 삭제, 결제, 고객 데이터 수정 같은 기능은 최소 권한 원칙과 승인 절차가 필요하다.

MCP 를 도입할 때는 “무엇을 연결할까”보다 “무엇을 연결하지 않을까”를 먼저 정하는 것이 안전하다.

실습 | MCP 연결 목록

회사에서 AI 가 읽어도 되는 시스템, 쓰기 가능한 시스템, 절대 연결하면 안 되는 시스템을 세 열로 나눈다. 각 시스템의 데이터 소유자와 승인자를 적는다.

체크리스트

- ☐ 읽기/쓰기 권한이 분리됐다
- ☐ 민감 시스템은 제외됐다
- ☐ 도구별 책임자가 있다
- ☐ 최신 사양 확인 절차가 있다

한 줄 정리

MCP 는 연결을 쉽게 만드는 표준이지만, 연결할 권한과 책임까지 대신 결정해 주지는 않는다.

CHAPTER 18

반복 업무를 스케줄과 조건으로 자동화하기

에이전트가 “필요할 때만” 움직이게 하면 AI가 일상 업무의 리듬 속으로 들어온다

세 가지 시작 방식

자동화는 보통 시간, 이벤트, 조건 중 하나로 시작한다. 매주 월요일 9시에 보고서를 만드는 것은 시간 기반, 새 이메일이 오면 분류하는 것은 이벤트 기반, 재고가 임계치 아래로 내려가면 알림을 보내는 것은 조건 기반이다.

시작 조건을 명확히 하면 필요 없는 실행을 줄이고 비용을 관리할 수 있다.

자동화는 관찰 가능해야 한다

백그라운드에서 돌아가는 시스템은 실패해도 사용자가 모를 수 있다. 마지막 실행 시간, 성공/실패, 처리 건수, 오류 메시지를 볼 수 있는 간단한 대시보드가 필요하다.

자동화가 조용히 실패하는 것보다 명시적으로 멈추고 알려주는 편이 낫다.

재시도와 중복 방지

같은 작업이 두 번 실행되면 메일이 중복 발송되거나 데이터가 중복 저장될 수 있다. 실행 ID, 처리 상태, 이미 처리한 항목 체크 같은 기본 장치가 필요하다.

외부 서비스가 잠시 실패했을 때는 일정 횟수 재시도하되, 반복 실패하면 사람에게 넘기는 규칙을 둔다.

비용과 빈도를 함께 설계한다

AI 호출은 실행 빈도가 늘수록 비용이 커진다. 매분 확인할 필요가 없는 업무를 매분 실행하지 않는다. 변화 속도와 의사결정 필요 시점에 맞춰 주기를 정한다.

자동화의 ROI는 절약 시간만이 아니라 누락 감소, 응답 속도, 오류율까지 함께 본다.

실습 | 자동화 카드 만들기

자동화 하나에 대해 Trigger / Input / AI 판단 / Action / Human Gate / Retry / Log / KPI를 한 장으로 정리한다.

체크리스트

- ☐ 중복 실행 방지가 있다
- ☐ 오류 알림이 있다
- ☐ 실행 로그가 있다
- ☐ 비용과 빈도를 측정한다

한 줄 정리

좋은 자동화는 “항상 돌아가는 것”이 아니라 필요한 순간에, 관찰 가능하게, 안전하게 움직이는 시스템이다.

CHAPTER 19

AI 품질·보안·거버넌스

회사에서 AI를 오래 쓰려면 정확도보다 책임 체계를 먼저 설계해야 한다

정확도는 하나의 숫자가 아니다

AI 품질은 업무마다 다르게 측정해야 한다. 분류 업무는 정확도와 누락률, 문서 생성은 사실 오류와 수정 시간, 리서치는 출처 적합성과 최신성, 영업은 후속물과 전환율이 중요하다.

“AI가 잘한다”는 감상 대신 업무 결과와 연결된 지표를 만든다.

개인정보와 기밀정보

AI 서비스에 데이터를 입력하기 전에 무엇이 개인정보이고 무엇이 영업기밀인지 분류해야 한다. 계약서, 급여, 고객 연락처, 미공개 가격 등은 조직 정책 없이 임의로 넣지 않는다.

가능하면 최소한의 필드만 사용하고, 익명화하거나 권한이 통제된 환경을 선택한다.

최소 권한과 분리

에이전트에게 필요한 권한만 준다. 읽기만 필요한 업무에 삭제 권한을 줄 이유가 없다. 테스트 환경과 운영 환경, 초안과 발송 권한도 분리한다.

권한이 클수록 승인과 감사가 강화되어야 한다.

거버넌스는 금지 목록이 아니다

AI 거버넌스의 목적은 사용을 막는 것이 아니라 안전하게 더 많이 사용할 수 있는 경계를 만드는 것이다. 허용 업무, 조건부 허용, 금지 업무를 구분하고, 위험도가 높은 업무의 승인 기준을 공개한다.

정책은 사용자의 실제 업무와 맞지 않으면 우회된다. 현업과 함께 만들고, 사고와 피드백을 반영해 계속 업데이트해야 한다.

실습 | AI 사용 정책 초안

팀에서 자주 하는 AI 사용 15 개를 허용 / 조건부 허용 / 금지로 분류하고 이유와 승인자를 적는다. 한 페이지를 넘기지 않게 만든다.

체크리스트

- ☐ 민감정보 기준이 있다
- ☐ 최소 권한 원칙이 있다
- ☐ 품질 KPI 가 업무별로 다르다
- ☐ 정책 업데이트 주기가 있다

한 줄 정리

거버넌스는 AI 를 막는 울타리가 아니라 조직이 자신 있게 사용할 수 있게 만드는 도로의 차선이다.

PART 7 90 일 안에 회사 AI 시스템 만들기

과제를 선택하고 90 일 안에 측정 가능한 성과로 전환하는 실행 계획을 완성한다.

CHAPTER 20

AI 전환 과제의 우선순위 정하기

모든 업무를 AI 로 바꾸려 하지 말고, 효과는 크고 위험은 낮은 곳부터 시작한다

ROI 와 실행 난이도의 2 축

AI 과제는 기대효과와 실행난이도로 나누어 보면 우선순위가 선명해진다. 효과가 크고 난이도가 낮은 과제는 빠른 성공을 만들고, 효과와 난이도가 모두 큰 과제는 장기 프로젝트로 관리한다.

효과는 시간 절약, 매출 기회, 오류 감소, 대응 속도, 직원 만족 등으로 나눠 평가한다.

데이터 준비도도 점수화한다

좋은 아이디어라도 필요한 데이터가 없거나 품질이 낮으면 바로 시작하기 어렵다. 데이터 위치, 구조화 정도, 최신성, 권한, 소유자를 점검한다.

데이터 준비도가 낮은 과제는 AI 프로젝트가 아니라 데이터 정비 프로젝트부터 해야 한다.

실패 비용을 반영한다

같은 2 시간 절약이라도 사내 요약과 계약서 자동화의 위험은 다르다. 실패 비용이 큰 업무는 더 강한 검증과 전문 검토가 필요하다.

첫 90 일에는 실패해도 되돌릴 수 있는 업무가 적합하다.

포트폴리오로 운영한다

한 개의 거대한 프로젝트보다 빠른 실험 2 개, 중간 규모 1 개, 장기 기반 1 개처럼 포트폴리오로 운영하면 학습과 성과를 동시에 얻을 수 있다.

각 프로젝트는 동일한 템플릿으로 목표, KPI, 데이터, 책임자, 다음 검토일을 기록한다.

실습 | AI 과제 점수표

후보 과제 10 개를 효과(1~5), 빈도(1~5), 데이터 준비도(1~5), 위험도(1~5), 난이도(1~5)로 평가한다. 위험도와 난이도는 낮을수록 가점을 주어 우선순위를 계산한다.

체크리스트

- ☐ KPI 가 숫자로 정의됐다
- ☐ 데이터 준비도를 평가했다
- ☐ 실패 비용이 반영됐다
- ☐ 90 일 안에 끝날 범위다

한 줄 정리

AI 전환은 기술 도입이 아니라 제한된 시간과 자원을 어디에 배치할지 정하는 포트폴리오 경영이다.

CHAPTER 21

90 일 실행 로드맵

3개월이면 “AI를 공부했다”가 아니라 실제 업무 하나가 작동하게 만들 수 있다

1~30 일: 관찰과 기준선

첫 달에는 시스템보다 업무를 본다. 후보 업무를 고르고 현재 처리시간, 오류, 대기시간, 재작업을 측정한다. 입력 자료와 승인 규칙을 정리하고, 사람이 직접 쓰는 체크리스트를 먼저 만든다.

AI를 붙이기 전 기준선이 있어야 효과를 증명할 수 있다.

31~60 일: MVP와 인간 승인

두 번째 달에는 가장 작은 end-to-end 흐름을 만든다. 데이터 한 종류, 사용자 한 명, 행동 한 가지로 제한하고 Human Gate를 넣는다.

매주 실제 사례를 처리하고 수정 내역을 기록한다. 기능을 늘리기보다 정확도와 사용 편의성을 먼저 높인다.

61~90 일: 운영과 확장 판단

세 번째 달에는 자동 실행 빈도, 실패율, 승인 수정률, 절약 시간, 사용자 만족을 측정한다. 운영 로그를 바탕으로 계속 쓸지, 범위를 넓힐지, 중단할지 결정한다.

성공했다면 두 번째 업무를 연결하되 공통 데이터와 권한 구조를 재사용한다.

90일 후 남아야 할 것

최종 산출물은 데모 영상이 아니다. 업무 설명서, 데이터 사전, 프롬프트 또는 규칙 버전, 테스트 사례, 승인 정책, 운영 로그, KPI 결과가 남아야 한다.

이 자산이 있어야 담당자가 바뀌어도 시스템을 이어갈 수 있다.

실습 | 90일 캘린더

주차별로 1주 문제정의, 2주 데이터정리, 3주 규칙, 4주 프로토타입, 5~6주 MVP, 7~8주 테스트, 9~10주 자동화, 11주 KPI 측정, 12주 운영결정으로 계획한다.

체크리스트

- ☐ 기준선이 있다
- ☐ MVP 범위가 작다
- ☐ 운영 KPI가 있다
- ☐ 중단 기준도 정의됐다

한 줄 정리

90일의 목표는 완벽한 AI가 아니라 한 업무에서 측정 가능한 개선과 재사용 가능한 운영 자산을 남기는 것이다.

CHAPTER 22

AI 시대의 리더십: 더 많이 일하지 말고 더 잘 설계하라

AI 가 업무 속도를 높일수록 리더의 역할은 지시에서 시스템 설계로 이동한다

답을 아는 리더에서 질문을 설계하는 리더로

AI 시대에는 정보를 가장 많이 가진 사람이 항상 우위에 있지 않다. 필요한 정보를 어떤 기준으로 찾고, 어떤 결정을 누구에게 맡길지 설계하는 능력이 중요해진다.

리더는 “이 보고서 만들어”보다 “이 결정에 필요한 증거와 반대 근거는 무엇인가”를 묻는 쪽으로 역할을 바꿔야 한다.

개인의 요령을 조직의 자산으로

한 사람이 AI 를 잘 쓰는 것은 시작일 뿐이다. 좋은 프롬프트, 체크리스트, 데이터 정의, 승인 규칙을 공유 자산으로 만들 때 조직이 강해진다.

성과가 난 워크플로는 문서화하고 교육하며, 실패 사례도 공개해 같은 실수를 줄인다.

사람에게 남겨야 할 일

문제 정의, 가치 판단, 관계 형성, 책임 있는 승인, 새로운 기회 발견은 여전히 사람의 핵심 역할이다. AI 가 초안을 만들수록 사람은 더 중요한 판단에 시간을 써야 한다.

AI 를 도입하고도 모두가 예전처럼 바쁘다면 자동화가 아니라 업무량만 늘어난 것일 수 있다. 줄일 일을 명시적으로 정해야 한다.

설계하는 조직

최종적으로 강한 조직은 특정 AI 제품에 의존하지 않는다. 데이터와 업무 기준, 권한, 평가 체계를 가지고 필요에 따라 모델과 도구를 교체한다.

기술은 계속 바뀌지만 잘 정의된 문제, 좋은 데이터, 명확한 책임, 측정 가능한 결과라는 경영 원칙은 바뀌지 않는다.

실습 | 나의 AI 운영 원칙 선언문

조직에서 지킬 AI 원칙을 7 문장으로 작성한다. 예: 사실은 원문과 대조한다 / 외부 발송은 사람이 승인한다 / 개인정보는 최소화한다 / 반복되는 수정은 규칙에 반영한다.

체크리스트

- ☐ AI 가 줄여야 할 업무가 정해졌다
- ☐ 공유 자산을 저장할 곳이 있다
- ☐ 사람에게 남길 판단이 명확하다
- ☐ 도구가 바뀌어도 유지될 원칙이 있다

한 줄 정리

AI 시대의 리더는 가장 많이 생성하는 사람이 아니라 사람과 AI 가 함께 일하는 시스템을 가장 잘 설계하는 사람이다.

부록 A 실무 템플릿

AI 업무 지시서

복사해서 쓰는 항목

목표 / 사용자 / 입력 / 판단 규칙 / 금지사항 / 출력형식 / 검증 / 예외 / 승인자 / 로그

사용 팁: 처음부터 모든 칸을 완벽하게 채우려 하지 말고, 실제 사례를 처리하면서 빈 칸을 보완한다. 문서가 길어지면 현업에서 쓰이지 않는다. 한 장 안에서 판단할 수 있는 수준을 유지한다.

에이전트 설계 캔버스

복사해서 쓰는 항목

Goal / Trigger / Inputs / Memory / Rules / Tools / Actions / Human Gate / Exit / KPI

사용 팁: 처음부터 모든 칸을 완벽하게 채우려 하지 말고, 실제 사례를 처리하면서 빈 칸을 보완한다. 문서가 길어지면 현업에서 쓰이지 않는다. 한 장 안에서 판단할 수 있는 수준을 유지한다.

데이터 사전

복사해서 쓰는 항목

필드명 / 정의 / 형식 / 단위 / 예시 / 결측처리 / 민감도 / 소유자 / 갱신주기

사용 팁: 처음부터 모든 칸을 완벽하게 채우려 하지 말고, 실제 사례를 처리하면서 빈 칸을 보완한다. 문서가 길어지면 현업에서 쓰이지 않는다. 한 장 안에서 판단할 수 있는 수준을 유지한다.

프롬프트 변경 기록

복사해서 쓰는 항목

버전 / 날짜 / 변경 이유 / 테스트셋 / 결과 / 회귀 오류 / 승인자

사용 팁: 처음부터 모든 칸을 완벽하게 채우려 하지 말고, 실제 사례를 처리하면서 빈 칸을 보완한다. 문서가 길어지면 현업에서 쓰이지 않는다. 한 장 안에서 판단할 수 있는 수준을 유지한다.

자동화 운영 카드

복사해서 쓰는 항목

주기 또는 이벤트 / 예상 처리량 / 재시도 / 중복방지 / 실패 알림 / 비용 / 담당자

사용 팁: 처음부터 모든 칸을 완벽하게 채우려 하지 말고, 실제 사례를 처리하면서 빈 칸을 보완한다. 문서가 길어지면 현업에서 쓰이지 않는다. 한 장 안에서 판단할 수 있는 수준을 유지한다.

AI 과제 평가표

복사해서 쓰는 항목

효과 / 빈도 / 데이터 준비도 / 위험도 / 난이도 / 예상 절감시간 / KPI / 90 일 가능성

사용 팁: 처음부터 모든 칸을 완벽하게 채우려 하지 말고, 실제 사례를 처리하면서 빈 칸을 보완한다. 문서가 길어지면 현업에서 쓰이지 않는다. 한 장 안에서 판단할 수 있는 수준을 유지한다.

부록 B 30 개의 실전 프로젝트 아이디어

1. 매일 아침 업계 뉴스 10 건을 수집해 회사 영향도와 다음 행동을 1 페이지로 요약
2. 정부·기관 지원사업을 수집해 자격조건과 마감일을 필터링하고 신청 링크까지 정리
3. 신규 문의 이메일을 견적/샘플/클레임/후속으로 분류하고 담당자에게 전달
4. 바이어 교신 이력을 기반으로 7 일 이상 무응답 대상을 찾아 후속메일 초안 생성
5. 제품 시험성적서와 스펙시트를 비교해 수치 불일치를 표시
6. 국가별 라벨 체크리스트와 제품 원문을 대조해 확인 필요 항목 표시
7. 월간 매출 CSV 를 정제하고 품목별·채널별·기간별 대시보드 생성
8. 재고가 임계치 아래로 내려간 SKU 를 감지해 구매 담당자에게 알림
9. 회의 녹취를 결정사항/담당자/기한으로 구조화해 태스크 목록 생성
10. 계약서 버전 두 개의 변경점을 조항별로 비교
11. 고객 FAQ 문서를 RAG 로 검색하는 사내 챗봇
12. 제품 카탈로그를 국가와 언어별로 자동 변환하고 금지 표현 검사
13. 경쟁사 20 곳의 신제품·가격·프로모션 변화를 주간 비교표로 작성
14. 전시회 명함 OCR 결과를 정리해 CRM 필드로 변환
15. 출장 일정과 미팅 주소를 연결해 일자별 동선 초안 생성
16. 교육자료를 장별 요약·퀴즈·실습 과제로 변환

17. 사내 SOP 를 읽고 신규 직원 온보딩 체크리스트 생성
18. 고객 리뷰를 긍정/부정/요청 기능으로 분류해 제품팀에 전달
19. 웹사이트 콘텐츠의 오래된 수치와 링크를 주기적으로 탐지
20. 블로그 글 한 편을 뉴스레터·SNS·FAQ 세 형식으로 재가공
21. 가격표 업데이트 시 이전 버전과 차이를 감지해 영업팀에 알림
22. 품질 클레임을 원인 유형으로 분류하고 반복 패턴을 월별 보고
23. 프로젝트 이슈를 읽고 주간 진행상황과 차주 리스크를 요약
24. 회사 공용 폴더에서 중복·구버전 파일 후보를 탐지
25. 제안서에 포함된 숫자의 원출처를 추적해 검증표 생성
26. 인보이스와 PO 의 품목·수량·가격 불일치 탐지
27. 직원별 AI 실습 과제를 자동 생성하고 제출 결과를 피드백
28. 고객 인터뷰 기록에서 반복되는 불편과 구매 장벽 추출
29. 신제품 아이디어를 시장성/제조난이도/규제위험 기준으로 비교
30. 대표의 일일 메모를 주간 의사결정 로그와 후속 행동으로 정리

참고자료와 더 읽을거리

KOTRA 글로벌 AI 리더 양성 과정 _AI 에이전트 실습 교육 Padlet
<https://padlet.com/jangpm/kotra-ai-ai-h42rpjcygukqhf21>

AI Ground, AI 에이전트 실습 교육 사례 - 법무법인 화현
<https://www.aiground.co.kr/hwahyeon-chatgpt-hands-on-practice/>

AI Ground, AI 에이전트 실습 교육 사례 - 대주전자재료
<https://www.aiground.co.kr/daejujeonjaeryo-chatgpt-hands-on-practice/>

OpenAI, Codex for knowledge work (2026)
<https://openai.com/index/codex-for-knowledge-work/>

OpenAI, Codex <https://openai.com/codex/>

Model Context Protocol 공식 문서 <https://modelcontextprotocol.io/>

MCP 2026-07-28 Specification 안 내
<https://blog.modelcontextprotocol.io/posts/2026-07-28/>

Google Codelabs, Gemini Function Calling
<https://codelabs.developers.google.com/codelabs/gemini-function-calling>

※ AI 제품과 사양은 빠르게 변경된다. 본문은 2026년 9월 기준의 개념과 공개 정보를 바탕으로 작성했으며, 실제 사용 시 각 서비스의 최신 공식 문서를 확인한다.

에필로그

AI에게 맡기고, 사람은 더 중요한 일을 한다

AI를 도입하는 가장 나쁜 방법은 사람의 모든 일을 그대로 복사해 기계에게 넘기는 것이다. 그러면 불필요한 절차도 더 빠르게 반복된다. 먼저 해야 할 일은 일을 줄이고, 기준을 정하고, 책임을 나누는 것이다.

좋은 AI 시스템은 인간의 판단을 제거하지 않는다. 오히려 어디에서 인간의 판단이 가장 가치 있는지 분명하게 만든다. 자료 찾기, 복사, 형식 맞추기, 첫 초안처럼 반복되는 일은 AI에게 맡기고, 사람은 문제를 정의하고 관계를 만들고 중요한 결정을 내리고 결과에 책임진다.

이 책에서 만든 템플릿과 체크리스트는 완성품이 아니다. 실제 업무에 적용하면서 바꿔야 한다. 한 번에 회사 전체를 바꾸려 하지 말고, 작은 업무 하나를 정확히 정의하고 90일 동안 개선해 보라. 그 경험이 두 번째 자동화의 기반이 되고, 결국 조직의 운영 방식을 바꾼다.

앞으로 가장 중요한 능력은 AI에게 무엇을 시킬 수 있는지를 많이 아는 것이 아니다. 무엇을 맡기고 무엇을 맡기지 않을지, 어떤 근거로 판단하고 어디에서 사람이 책임질지를 설계하는 능력이다. AI 시대의 경쟁력은 결국 “일을 설계하는 힘”으로 돌아온다.